ARTEFACTOS DEL MÓDULO 6

Punto de Control 3 (PC3)

Fase de Elaboración: arquitectura, contratos y diseño del módulo

Pacíficos Online — IF6100

INFORMACIÓN DEL SUBGRUPO

Subgrupo y módulo	*SG6 — M6 — Pagos y Facturación Electrónica*
Integrantes	*Ian Moraga - C35281 - Diseñador/Modelador* *Anthony Masis - C24619 - Implementador Líder* *Félix Luna - C34434 - Ingeniero de Pruebas/QA* *Juan José Murillo - C35485 - Analista de Requerimientos*
Representante ante el Comité	*Justin Marenco Herrera*
Fecha de entrega	Jueves 22 de octubre de 2026 (Semana 11)
Versión del documento	*1*

INSTRUCCIONES

- Este documento es la plantilla común para los 7 subgrupos. Cada subgrupo hace su copia y la nombra SG#_M#_Artefactos_PC3.
- Los textos en gris entre corchetes [así] se sustituyen por el contenido del módulo. Los recuadros azules «Cómo hacerlo» explican qué se espera; se eliminan en la versión final.
- La sección 1 se completa primero: define de qué depende el módulo, tanto de otros módulos como de los artefactos del Comité.
- Los diagramas se insertan como imagen y su archivo fuente se guarda en el repositorio.
- Todo lo que el documento menciona como código (contrato, migraciones, dobles de prueba, prototipo) debe existir en el repositorio único.

1. Dependencias del módulo

Antes de diseñar, el subgrupo deja claro qué necesita de otros módulos, qué les ofrece y qué toma de los artefactos del Comité. Esta sección alimenta el contrato (sección 2), los diagramas de interacción (sección 5) y los dobles de prueba (sección 9).

1.1 Dependencias con los artefactos del Comité (PC3)

Cada artefacto del subgrupo usa algo que define el Comité. Si el Comité aún no lo ha publicado, el subgrupo anota el supuesto que usa y lo ajusta cuando esté disponible.

Artefacto del subgrupo	Depende de (Comité PC3)	Qué toma de ahí	Estado
2. Contrato OpenAPI	4.1 Reglas de publicación de contratos; carpeta contratos/ del repositorio; 5.5 Manejo de errores	Formato, esquema de versionado, ubicación del archivo, estructura estándar de error	Completado
3 y 4. Realización y clases de diseño	1.5 Vista lógica (capas); 5. Núcleo compartido	Capas internas que siguen todos los módulos; clases del núcleo para identidad, roles y aislamiento	Completado
5. Diagramas de interacción	4.2 y 4.3 Contratos de otros módulos; 1.6 Vista de procesos	Operaciones y eventos disponibles; qué interacciones son síncronas o por eventos	Completado
6. Modelo de datos y migraciones	6. Modelo de datos consolidado; 1.8 Convenciones de implementación; 1.9 Decisiones de datos	Tablas reservadas, claves compartidas, convención de nombres, uso o no de llaves foráneas entre módulos	Completado
7. Patrones de diseño	3. ADR	Decisiones transversales que los patrones del módulo no deben contradecir	Completado
8. Prototipo de interfaz	ADR del frontend; carpeta frontend/ y componentes compartidos	React + Tailwind CSS, organización por módulo, componentes comunes	Completado
9. Dobles de prueba	4. Contratos publicados de otros módulos	Versión del contrato que el doble debe imitar	Completado

1.2 Lo que el subgrupo entrega al Comité

El Comité necesita estos insumos para cerrar sus artefactos. El representante ante el Comité es el responsable de entregarlos.

Insumo	Destino en el Comité	Fecha	Entregado
Lista de tablas corregida del módulo	6. Modelo de datos consolidado	Jue 1 oct	Sí
Borrador del contrato (operaciones y eventos)	4. Contratos publicados	Sáb 3 oct	Sí
Contrato OpenAPI en la carpeta contratos/	4. Contratos publicados	Sáb 17 oct	Sí
Módulo desplegado en el ambiente (implementación mínima)	7. Ambiente de integración	Miér 21 oct	Sí

2. Contrato formal del módulo (OpenAPI)

Descripción formal de las operaciones y eventos que el módulo ofrece, versionada y publicada en el repositorio.

Cómo hacerlo

- Escribir el contrato en formato OpenAPI (YAML) siguiendo las reglas de publicación del Comité (4.1) y usar el formato de error estándar del núcleo (5.5).
- Usar las claves compartidas con el formato del modelo canónico (id_entidad).
- Validar el archivo en un editor de OpenAPI antes de publicarlo y subirlo a la carpeta contratos/ del módulo.
- Un cambio que rompe a los consumidores genera una versión nueva; no se modifica la versión ya publicada.

2.1 Datos del contrato

Versión	v1.0.0
Archivo en el repositorio	contratos/m6-pagos-facturacion.yaml
Versión de OpenAPI	3.0.3
Estado	Publicado
Consumidores	M1 (Portal público y cliente), M2 (Micrositios y comercios), M5 (Carrito y gestión de pedidos), M7 (ERP contable)

2.2 Operaciones

Método	Ruta	Descripción	Entrada	Respuesta exitosa	Errores
GET	/health	Verificación de estado del módulo	Ninguna	200 OK: Object {status: "ok", module: "m6-pagos"}	N/A
POST	/pagos	Procesar el pago de un pedido confirmado (RF-001, CU-001, RN-001)	**Header:** Idempotency-Key (string) **Body:** {id_pedido: uuid, medio_pago: enum[tarjeta, sinpe, paypal], monto_total: decimal}	201 Created: Pago	400 Bad Request (MONTO_INCONSISTENTE) 404 Not Found (PEDIDO_NO_ENCONTRADO) 422 Unprocessable Entity (MEDIO_PAGO_NO_DISPONIBLE)
GET	/pagos/{id_pago}	Consultar el estado actual de un intento de pago	**Path:** id_pago (uuid)	200 OK: Pago	404 Not Found (PAGO_NO_ENCONTRADO)
POST	/pagos/{id_pago}/confirmaciones	Registrar la confirmación de cobro enviada por el proveedor externo (RF-002, CU-002, RN-002)	**Path:** id_pago (uuid) **Body:** {referencia_proveedor: string, estado: enum[aprobado, rechazado]}	200 OK: {id: uuid, estado: enum[confirmado, rechazado], fecha: date-time}	401 Unauthorized (NOTIFICACION_NO_AUTENTICA) 404 Not Found (TRANSACCION

Método	Ruta	Descripción	Entrada	Respuesta exitosa	Errores
					_NO_ENCONTRADA)
POST	/pedidos/{id_pedido}/reembolsos	Solicitar reembolso total o parcial de un pedido pagado (RF-007, CU-007, RN-006)	**Path:** id_pedido (uuid) **Body:** {id_subpedido?: uuid, monto: decimal, motivo: string}	201 Created: Reembolso	409 Conflict (PAGO_NO_CONFIRMADO) 422 Unprocessable Entity (REEMBOLSO_EXCEDE_MONTO)
GET	/facturas	Listar facturas electrónicas generadas para un pedido	**Query: id_pedido (uuid, requerido)**	200 OK: Array de Factura	N/A
GET	/facturas/{id_factura}	Obtener el documento de una factura electrónica (RF-004, CU-004, RN-003)	**Path:** id_factura (uuid)	200 OK: Factura	404 Not Found (FACTURA_NO_ENCONTRADA)
POST	/facturas/{id_factura}/notas-credito	Generar una nota de crédito sobre una factura existente (RF-005, CU-005, RN-004)	**Path:** id_factura (uuid) **Body:** {monto: decimal, motivo: string}	201 Created: NotaCreditoDebito	404 Not Found (FACTURA_NO_ENCONTRADA) 422 Unprocessable Entity (NOTA_EXCEDE_MONTO_FACTURA)
POST	/facturas/{id_factura}/notas-debito	Generar una nota de débito sobre una factura existente (RF-006, CU-006, RN-005)	**Path:** id_factura (uuid) **Body:** {monto: decimal, motivo: string}	201 Created: NotaCreditoDebito	404 Not Found (FACTURA_NO_ENCONTRADA) 409 Conflict (FACTURA_NO_ACEPTADA)
POST	/conciliaciones	Conciliar pagos del periodo contra reportes de pasarelas y M7 (RF-003, CU-003, RN-007)	**Body:** {fecha_inicio: date, fecha_fin: date}	200 OK: ReporteConciliacion	422 Unprocessable Entity (PERIODO_INVALIDO)
GET	/historial	Consultar historial de pagos y facturas con aislamiento por comercio (RF-008, CU-008, RN-008)	**Query:** id_comercio? (uuid), id_cliente? (uuid)	200 OK: {pagos: Array<Pago>, facturas: Array<Factura>}	404 Not Found (SIN_RESULTADOS)

2.3 Eventos publicados

Evento	Versión	Cuándo se publica	Consumidores	Carga útil (campos y tipos)

PagoConfirmado	v1	Se publica al invocar /pagos/{id_pago}/confirmaciones solo si el estado recibido es "aprobado" y el intento de pago estaba en estado "pendiente".	M5, M7	{id_pedido: uuid, id_pago: uuid, monto: decimal, fecha: date-time}
ReembolsoProcesado	v1	Se publica inmediatamente tras procesar y aceptar la solicitud de reembolso total o parcial en /pedidos/{id_pedido}/reembolsos.	M5, M7	{id_pedido: uuid, id_reembolso: uuid, monto: decimal, nota_credito_id: uuid, fecha: date-time}

2.4 Esquemas expuestos

Pago: Representa el intento de cobro registrado sobre un pedido.

- id (uuid, formato UUID)
- id_pedido (uuid)
- medio_pago (string, enum: [tarjeta, sinpe, paypal])
- monto_total (number, format: decimal)
- estado (string, enum: [pendiente, aprobado, rechazado, confirmado])
- fecha_creacion (string, format: date-time)
- fecha_confirmacion (string, format: date-time, nullable)

Reembolso: Detalle de la solicitud de reembolso.

- id (uuid)
- id_pedido (uuid)
- id_subpedido (uuid, nullable)
- monto (number, format: decimal)
- motivo (string)
- estado (string, enum: [aceptado, rechazado, procesado])
- nota_credito_id (uuid, nullable)
- fecha (string, format: date-time)

Factura: Documento de comprobante electrónico emitido por subpedido/comercio.

- id (uuid)
- id_pedido (uuid)
- id_subpedido (uuid)
- id_comercio (uuid)
- consecutivo (string)
- xml (string: comprobante en formato XML)
- estado_hacienda (string, enum: [pendiente, aceptada, rechazada])

- fecha_emision (string, format: date-time)

NotaCreditoDebito: Documento fiscal de ajuste emitido sobre una factura existente.

- id (uuid)
- id_factura (uuid)
- tipo (string, enum: [credito, debito])
- monto (number, format: decimal)
- motivo (string)
- xml (string: documento XML)
- fecha_emision (string, format: date-time)

ReporteConciliacion: Resultado de la conciliación de pagos del periodo.

- fecha_inicio (string, format: date)
- fecha_fin (string, format: date)
- total_cobrado (number, format: decimal)
- total_reembolsado (number, format: decimal)
- estado (string, enum: [completa, parcial])
- diferencias (array de objetos con id_pago, monto_sistema, monto_pasarela, estado: [pendiente_de_revision, resuelto])

ErrorResponse: Estructura del objeto de respuesta ante errores.

- codigo (string)
- mensaje (string)

PagoConfirmadoEvent: Payload del evento publicado al confirmarse un pago.

- id_pedido (uuid)
- id_pago (uuid)
- monto (number, format: decimal)
- fecha (string, format: date-time)

ReembolsoProcesadoEvent: Payload del evento publicado al procesarse un reembolso.

- id_pedido (uuid)
- id_reembolso (uuid)
- monto (number, format: decimal)
- nota_credito_id (uuid)
- fecha (string, format: date-time)

2.5 Historial de versiones del contrato

Versión	Fecha	Cambio	¿Rompe consumidores?
v1.0.0	21/10/2026	Publicación inicial de la especificación de la API y eventos para M6.	No

3. Realización de casos de uso (mínimo dos)

Muestra cómo las clases del módulo cumplen un caso de uso, separando clases límite, control y entidad.

Cómo hacerlo

- Elegir al menos dos casos de uso críticos especificados en el PC2; preferir los que involucran a otros módulos.
- Clase límite: lo que interactúa con el exterior (pantalla React, controlador de la API, cliente de otro módulo). Clase control: coordina la lógica del caso de uso. Clase entidad: datos que el módulo guarda.
- Dibujar un diagrama de clases de análisis y un diagrama de comunicación o secuencia del flujo principal.
- Las clases que aparecen aquí deben reaparecer, refinadas, en el diagrama de clases de diseño (sección 4).

3.1 Realización del caso de uso [CU-001: Procesar pago del pedido]

Caso de uso (PC2)	*CU-001: Procesar pago del pedido*
Actor principal	*Comprador*
Módulos externos involucrados	*M5: ConsultarEstadoPedido y ObtenerDatosPedido (consultas síncronas) y NotificarResultadoPago (resultado del cobro)*

Estereotipo	Clase	Responsabilidad en este caso de uso
Límite	FormularioPagoUI	Pantalla donde el comprador elige el medio de pago y confirma el monto. Muestra el resultado del pago o el error
Límite	AdaptadorM5	Interfaz con M5 según el contrato. Consulta el estado del pedido, obtiene sus datos (monto total, desglose por subpedido, cliente) y notifica el resultado del pago.
Límite	AdaptadorProveedorPago	Interfaz con el proveedor de pago externo. Envía la solicitud de cobro con clave de idempotencia y token del medio de pago (nunca datos de tarjeta), recibe la referencia y el estado inicial, y detecta el timeout.
Control	ProcesarPagoControlador	Orquesta el caso de uso. Consulta a M5, valida la solicitud, registra el pago pendiente antes de cobrar, solicita el cobro, registra la respuesta, notifica a M5 y programa el reintento si el proveedor no responde.
Entidad	Pago	Registra el intento de pago: pedido, medio, monto, estado y referencia_proveedor. Se crea en estado pendiente y se actualiza con la respuesta del proveedor.
Entidad	MedioPago	Catálogo de medios de pago. Permite verificar que el medio elegido está disponible.
[Estereotipo]	[Clase]	[Responsabilidad]

Por temas de espacio no se puede adjuntar el diagrama en este documento, pero se colocó en la siguiente dirección: pacífico_online/docs/actas-subgrupos/SG6_Pagos_Y_Facturacion_Electronica/Diagramas_PC3

3.2 Realización del caso de uso [CU-002: Confirmar pago recibido]

Caso de uso (PC2)	CU-002: Confirmar pago recibido
Actor principal	Proveedor de pago externo (pasarela de pago, SINPE, PayPal)

Módulos externos involucrados	M5: evento PagoConfirmado (y PagoRechazado en el rechazo tardío). M7: evento financiero, mediante el caso de uso incluido CU-010.

Estereotipo	Clase	Responsabilidad en este caso de uso
Límite	WebhookProveedorPago	Punto de entrada que recibe la notificación del proveedor (identificador del intento, referencia, estado y firma) y devuelve la respuesta de recepción.
Límite	AdaptadorEventos	Publica hacia M5 los eventos PagoConfirmado y PagoRechazado.
Límite	AdaptadorM7	Interfaz con M7 para entregar el evento financiero (lo usa CU-010).
Control	ConfirmarPagoControlador	Orquesta el caso de uso. Valida la autenticidad de la notificación, localiza el pago, aplica la idempotencia (RN-M6-02), confirma o rechaza el pago, publica el evento correspondiente e invoca CU-010.
Control	NotificarEventoFinancieroControlador	Realiza el caso de uso incluido CU-010: prepara y envía el evento financiero a M7.
Entidad	NotificacionPago	Registra cada aviso recibido. Garantiza la idempotencia mediante una restricción de unicidad sobre referencia_proveedor y guarda las incidencias (intento inexistente).
Entidad	Pago	Intento de pago que se confirma o rechaza. Los cambios de estado se hacen solo si el pago sigue pendiente (confirmarSiPendiente, rechazarSiPendiente), para evitar confirmaciones duplicadas.
[Estereotipo]	[Clase]	[Responsabilidad]

Por temas de espacio no se puede adjuntar el diagrama en este documento, pero se colocó en la siguiente dirección: pacífico_online/docs/actas-subgrupos/SG6_Pagos_Y_Facturacion_Electronica/Diagramas_PC3

4. Diagrama de clases de diseño del módulo

Diagrama completo del módulo listo para programar: con tipos, visibilidad, firmas de métodos y navegabilidad.

Cómo hacerlo

- Refinar las clases de análisis de la sección 3 hasta el nivel de implementación en Laravel.
- Tipos: cada atributo y parámetro con su tipo (int, string, decimal, bool, fecha, enumeración, otra clase).
- Visibilidad: + pública, - privada, # protegida.
- Firmas: nombre del método, parámetros con tipo y tipo de retorno. Ejemplo de forma: + nombreMetodo(param: Tipo): TipoRetorno.
- Navegabilidad: flechas que indican qué clase conoce a cuál; incluir multiplicidades.
- Respetar las capas de la vista lógica del Comité (1.5) y usar las clases del núcleo para identidad, roles y aislamiento en lugar de reimplementarlas.

Debido a lo grande de el diagrama, se puede encontrar mejor en pacifico-online/docs/actas-subgrupos/SG6_Pagos_Y_Facturacion_Electronica/Diagramas_PC3

4.1 Resumen de clases

Clase	Capa	Atributos principales	Métodos principales (firma)	Relaciones y navegabilidad
FormularioPagoUI «boundary»	Presentación	—	+ seleccionarMedioPago(idPedido: Long, idMedioPago: Long): void; + mostrarResultado(): void; + mostrarError(): void; + ofrecerOtroMedioPago(): void; + mostrarPagoEnProceso(): void	Conoce a ProcesarPagoControlador (1 → 1)
WebhookProveedorPago «boundary»	Presentación	—	+ notificarResultadoPago(idIntentoPago: Long, referenciaProveedor: String, estadoNotificado: String, firma: String): void	Conoce a ConfirmarPagoControlador (1 → 1)

ProcesarPagoControlador	Aplicación	- pedidoClient: PedidoClient; - pasarelaFactory: PasarelaPagoFactory	+ procesarPago(idPedido: Long, idMedioPago: Long, montoTotal: Decimal, claveIdempotencia: String): Pago; - validarSolicitud(estadoPedido: String, datosPedido: PedidoDTO, medioPago: MedioPago, montoTotal: Decimal): void; - programarReintento(idPago: Long): void	Conoce a PedidoClient (1 → 1) y a PasarelaPagoFactory (1 → 1). Depende de Pago y MedioPago
ConfirmarPagoControlador	Aplicación	- eventPublisher: EventPublisher; - notificarEventoControlador: NotificarEventoFinancieroControlador	+ confirmarPago(idIntentoPago: Long, referenciaProveedor: String, estadoNotificado: String, firma: String): void; - validarAutenticidad(firma: String): Boolean	Conoce a EventPublisher (1 → 1). Depende de NotificarEventoFinancieroControlador («include», CU-010), Pago y NotificacionPago
NotificarEventoFinancieroControlador	Aplicación	- contabilidadClient: ContabilidadClient	+ notificarEventoFinanciero(idPago: Long, tipo: String): void	Conoce a ContabilidadClient (1 → 1)
ReembolsoService	Aplicación	- pasarelaFactory: PasarelaPagoFactory; - eventPublisher: EventPublisher; - facturacionService: FacturacionService	+ gestionarReembolso(idPedido: Long, idSubpedido: Long, monto: Decimal, motivo: String): Reembolso	Conoce a PasarelaPagoFactory, EventPublisher y FacturacionService (1 → 1 cada una). Depende de Reembolso y Pago
FacturacionService	Aplicación	- haciendaGateway: HaciendaGateway; - comercioClient: ComercioClient	+ generarFacturaElectronica(idPedido: Long, subpedidos: List<SubpedidoDTO>): List<Factura>; + generarNotaCredito(idFactura: Long, monto: Decimal,	Conoce a HaciendaGateway y ComercioClient (1 → 1 cada una). Depende de Factura,

			motivo: String): NotaCredito; + generarNotaDebito(idFactura: Long, monto: Decimal, motivo: String): NotaDebito	NotaCredito y NotaDebito
ConciliacionService	Aplicación	- contabilidadClient: ContabilidadClient	+ conciliarPeriodo(fechaInicio: Date, fechaFin: Date): ReporteConciliacion	Conoce a ContabilidadClient (1 → 1). Depende de ReporteConciliacion
HistorialService	Aplicación	—	+ consultarHistorial(idComercio: Long, idCliente: Long): HistorialDTO	Depende de Pago y Factura
Pago	Dominio	- idPago: Long; - idPedido: Long; - idCliente: Long; - idMedioPago: Long; - estado: EstadoPago; - monto: Decimal; - claveIdempotencia: String; - referenciaProveedor: String	+ {static} crear(idPedido: Long, idCliente: Long, idMedioPago: Long, monto: Decimal, claveIdempotencia: String): Pago; + {static} buscar(idPago: Long): Pago; + registrarRespuestaProveedor(referenciaProveedor: String, estadoInicial: String): void; + confirmarSiPendiente(idPago: Long): Pago; + rechazarSiPendiente(idPago: Long): Pago; + esConfirmado(): Boolean	Asociada con Factura (1 — 1.*., ambos sentidos). Conoce a MedioPago (0.. → 1). Es conocida por NotificacionPago, Reembolso y DiferenciaConciliacion (0..* → 1)
MedioPago	Dominio	- idMedioPago: Long; - nombreMedio: String	+ {static} obtenerSiDisponible(idMedioPago: Long): MedioPago	Es conocida por Pago y NotificacionPago (0..* → 1)
NotificacionPago	Dominio	- idNotificacionPago: Long; - idPago: Long; -	+ {static} registrarSiNoExiste(pago: Pago, referenciaProveedor: String, estadoNotificado:	Conoce a Pago (0..* → 1) y a MedioPago (0..* → 1)

		idMedioPago: Long; - estado: EstadoNotificacionPago; - referenciaProveedor: String; - fechaRecepcion: DateTime	String): ResultadoRegistro; + {static} registrarIncidencia(referenciaProveedor: String, estado: String): void	
Reembolso	Dominio	- idReembolso: Long; - idPago: Long; - idSubpedido: Long; - idNotaCredito: Long; - estado: EstadoReembolso; - monto: Decimal; - motivo: String	+ procesar(): void; + excedeMonto(montoDisponible: Decimal): Boolean	Conoce a Pago (0..* → 1) y a NotaCredito (0..1 → 0..1)
Factura	Dominio	- idFactura: Long; - idPago: Long; - idSubpedido: Long; - idComercio: Long; - idCliente: Long; - estadoHacienda: EstadoHacienda; - consecutivo: String; - xml: String; - montoTotal: Decimal; - razonSocialEmisor: String; - cedulaJuridicaEmisor: String; - identificacionReceptor: String	+ generarXml(): String; + marcarAceptada(): void; + marcarRechazada(motivo: String): void	Asociada con Pago (1..* — 1, ambos sentidos). Es conocida por DocumentoAjuste (0..* → 1)
DocumentoAjuste «abstracta»	Dominio	# id: Long; # idFactura: Long; # monto: Decimal; # motivo: String; # xml: String; # fechaEmision: DateTime	+ generarXml(): String; + {abstract} validarMonto(facturaOriginal: Factura): Boolean; + emitir(facturaOriginal: Factura): void	Conoce a Factura (0..* → 1). Es la superclase de NotaCredito y NotaDebito

NotaCredito	Dominio	Hereda los de DocumentoAjuste	+ validarMonto(facturaOriginal: Factura): Boolean (RN-004: el monto no supera el total de la factura)	Hereda de DocumentoAjuste. Es conocida por Reembolso (0..1 → 0..1)
NotaDebito	Dominio	Hereda los de DocumentoAjuste	+ validarMonto(facturaOriginal: Factura): Boolean (RN-005: la factura está aceptada por Hacienda)	Hereda de DocumentoAjuste
ReporteConciliacion	Dominio	- idConciliacion: Long; - fechaInicio: Date; - fechaFin: Date; - totalCobradoSistema: Decimal; - totalReportadoPasarela: Decimal; - estado: EstadoConciliacion; - fechaGeneracion: DateTime	+ agregarDiferencia(d: DiferenciaConciliacion): void; + calcularTotales(): void	Compone DiferenciaConciliacion (1 ◆— 0..*)
DiferenciaConciliacion	Dominio	- idDiferencia: Long; - idPago: Long; - tipo: TipoDiferencia; - estado: EstadoRevisionDiferencia; - montoDiferencia: Decimal	+ marcarResuelto(): void	Es parte de ReporteConciliacion (composición). Conoce a Pago (0..* → 1)

Enumeraciones (8)	Dominio	EstadoPago {PENDIENTE, CONFIRMADO, RECHAZADO}; EstadoNotificacionPago {RECIBIDA, PROCESADA, DUPLICADA, RECHAZADA}; EstadoReembolso {SOLICITADO, PROCESADO, RECHAZADO}; EstadoHacienda {PENDIENTE_ENVIO, ENVIADA, ACEPTADA, RECHAZADA}; EstadoConciliacion {EN_PROCESO, PARCIAL, COMPLETADA}; EstadoRevisionDiferencia {PENDIENTE_REVISION, RESUELTA}; TipoDiferencia {MONTO_DISTINTO, FALTANTE_EN_PASARELA, FALTANTE_EN_SISTEMA}; ResultadoRegistro {REGISTRADA, DUPLICADA}	—	Sirven de tipo a los atributos de las entidades; ResultadoRegistro es el retorno de registrarSiNoExiste. No llevan flechas
PasarelaPagoFactory	Infraestructura	—	+ obtener(nombreMedio: String): PasarelaPagoGateway	Conoce a PasarelaPagoGateway (1 → 1). La usan ProcesarPagoControlador y ReembolsoService

PasarelaPagoGateway «interface»	Infraestructura	—	+ solicitarCobro(idPago: Long, montoTotal: Decimal, tokenMedioPago: String): ResultadoCobro; + reembolsar(referenciaProveedor: String, monto: Decimal): ResultadoCobro	Realizada por AdaptadorTarjeta, AdaptadorSinpe y AdaptadorPayPal
AdaptadorTarjeta, AdaptadorSinpe, AdaptadorPayPal	Infraestructura	—	Implementan solicitarCobro y reembolsar con la misma firma de la interfaz	Realizan PasarelaPagoGateway
PedidoClient «interface»	Infraestructura	—	+ consultarEstadoPedido(idPedido: Long): String; + obtenerDatosPedido(idPedido: Long): PedidoDTO; + notificarResultadoPago(idPedido: Long, idPago: Long, estado: String, motivo: String): void	Realizada por AdaptadorM5. La usa ProcesarPagoControlador
AdaptadorM5	Infraestructura	—	Implementa los tres métodos de PedidoClient con la misma firma	Realiza PedidoClient
ContabilidadClient «interface»	Infraestructura	—	+ enviarEventoFinanciero(idPago: Long, tipo: String): void; + consultarRegistroContablePago(idPago: Long): Boolean	Realizada por AdaptadorM7. La usan NotificarEventoFinancieroControlador y ConciliacionService
AdaptadorM7	Infraestructura	—	Implementa los dos métodos de ContabilidadClient con la misma firma	Realiza ContabilidadClient

EventPublisher «interface»	Infraestructura	—	+ publicarPagoConfirmado(idPedido: Long, idPago: Long, monto: Decimal): void; + publicarPagoRechazado(idPedido: Long, idPago: Long, motivo: String): void; + publicarReembolsoProcesado(idPedido: Long, idSubpedido: Long, idComercio: Long, idReembolso: Long, monto: Decimal, idNotaCredito: Long): void	Realizada por AdaptadorEventos. La usan ConfirmarPagoControlador y ReembolsoService
AdaptadorEventos	Infraestructura	—	Implementa los tres métodos de EventPublisher con la misma firma	Realiza EventPublisher
ComercioClient «interface»	Infraestructura	—	+ obtenerDatosFiscales(idComercio: Long): DatosFiscalesDTO	La usa FacturacionService
HaciendaGateway «interface»	Infraestructura	—	+ transmitirComprobante(xml: String): EstadoTransmision	La usa FacturacionService

4.2 Clases del núcleo y de otros módulos que se usan

Clase o interfaz externa	Origen	Para qué la usa el módulo
Trait BelongsToComercio (Global Scope de Eloquent)	Núcleo (ADR-006)	Se aplica al modelo de facturas_electronicas, que tiene id_comercio, para que ninguna consulta devuelva facturas de otro comercio (RN-008).

Middleware de autenticación, roles y permisos	Núcleo	Identifica al comprador y al administrador del comercio, y limita al administrador la conciliación, las notas de crédito y débito y el historial.
Bitácora de auditoría	Núcleo	Deja registro de las operaciones financieras sensibles: reembolsos, notas de crédito y débito y conciliaciones.
Formato uniforme de respuesta y de errores	Núcleo	Devuelve los errores de M6, como MONTO_INCONSISTENTE, en el formato común de la plataforma.
Bus de eventos	Plataforma (ADR-005)	Por él, AdaptadorEventos publica PagoConfirmado y ReembolsoProcesado hacia M5 y M7.
consultarEstadoPedido, obtenerDatosPedido, PedidoDTO y SubpedidoDTO	Cliente del contrato de M5 (PedidoClient y AdaptadorM5)	Verificar que el pedido esté confirmado y obtener monto total, desglose por subpedido y cliente antes de cobrar y de facturar.
consultarRegistroContablePago	Cliente del contrato de M7 (ContabilidadClient y AdaptadorM7)	Confirmar que el asiento y la cuenta por pagar del pago quedaron registrados al conciliar el periodo (CU-003).
obtenerDatosFiscales y DatosFiscalesDTO	Cliente del contrato de M2 (ComercioClient)	Obtener la razón social y la cédula jurídica del comercio emisor para generar la factura y guardar su copia fiscal.

5. Diagramas de interacción actualizados

Diagramas de secuencia de los casos de uso del PC2 actualizados con las clases de diseño, incluyendo las llamadas a otros módulos.

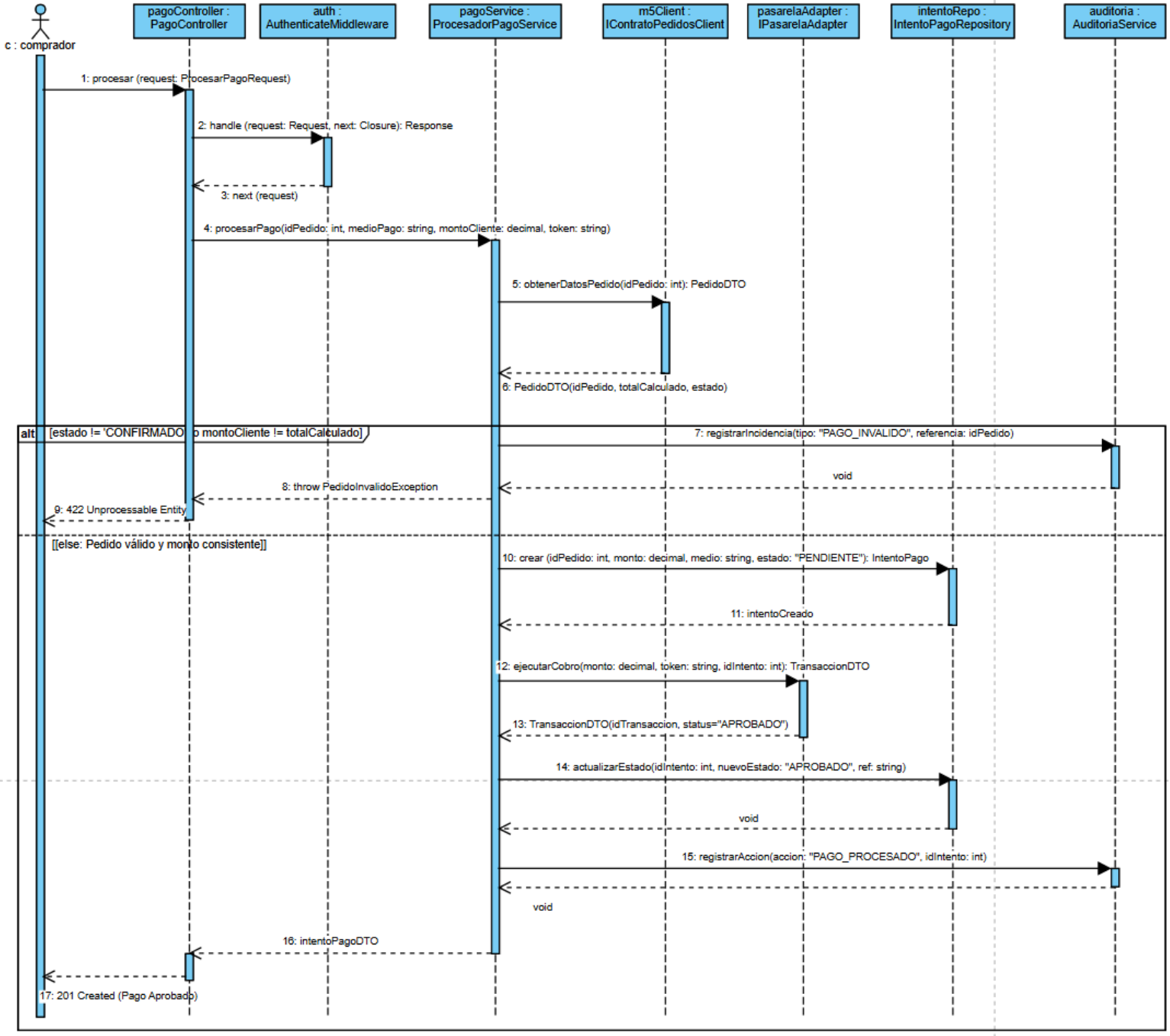
Cómo hacerlo

- Tomar los diagramas de interacción del PC2 y reemplazar los objetos de análisis por las clases de diseño de la sección 4.
- Cada mensaje debe corresponder a un método que exista en el diagrama de clases, con su firma.
- Las llamadas a otro módulo se muestran a través de su contrato (operación o evento), nunca accediendo a sus tablas o clases internas.
- Mostrar también las llamadas al núcleo (autenticación, verificación de permisos, bitácora) y los flujos de error relevantes.

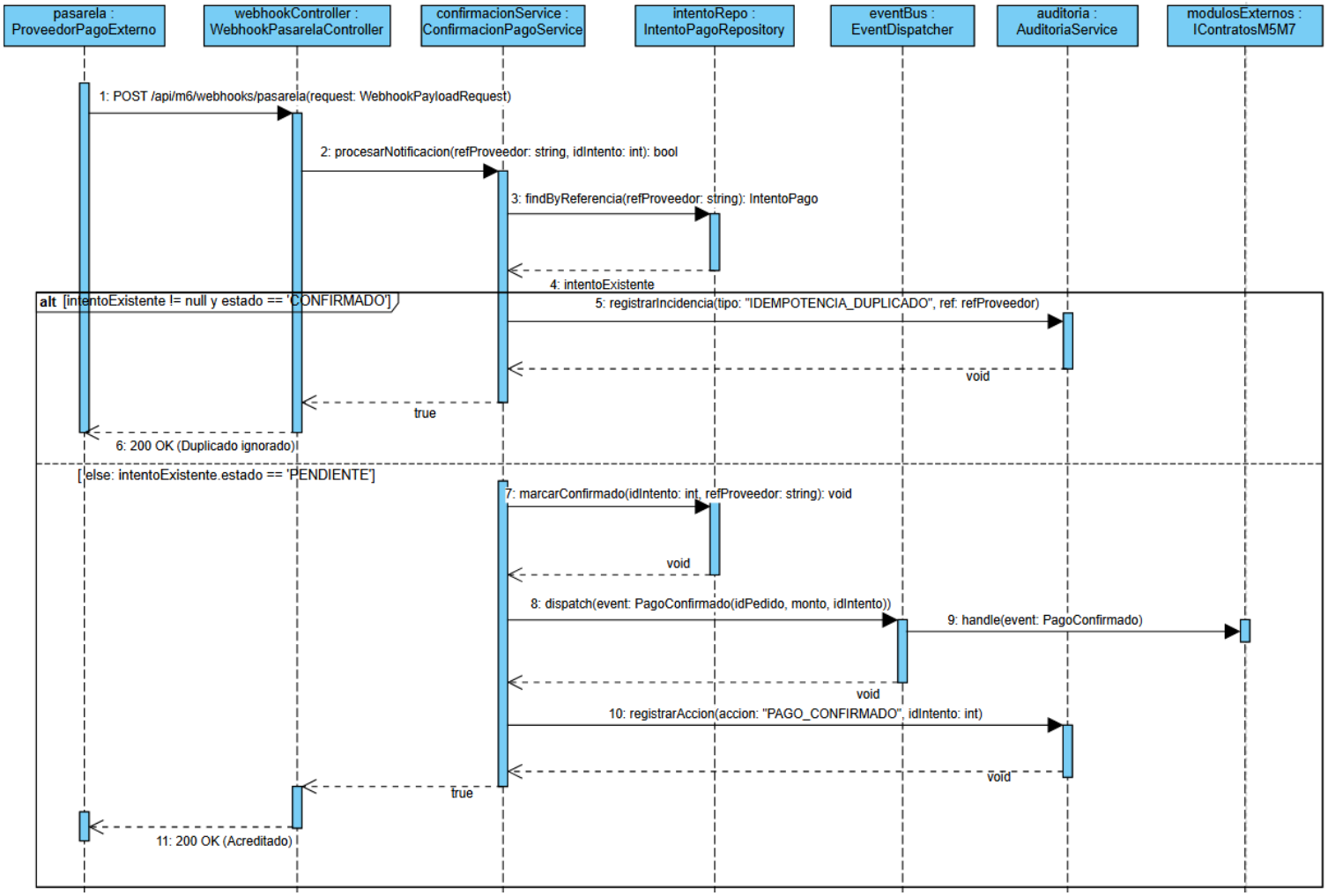
Diagrama	Caso de uso	Módulos externos llamados	Operación o evento del contrato usado
DS-01	CU-001: Procesar pago del pedido	- Módulo 5 (Carrito y gestión de pedidos)	M5/IContratoPedidos:: obtenerDatosPedido (id_pedido)
DS-02	CU-002: Confirmar pago recibido e Idempotencia	- Módulo 5 (Carrito y gestión de pedidos - Módulo 7 (ERP contable)	Evento publicado: M6\Events\PagoConfirmado (id_pedido, monto, id_intento) (consumido por M5 y M7)
DS-03	CU-004: Generar factura electrónica XML	- Módulo 5 (Carrito y gestión de pedidos) - Módulo 2 (Micrositios y comercios) - Módulo 1 (Portal público y cliente) - Módulo 7 (ERP contable)	- Evento consumido: M5\Events\PedidoListoParaFacturar(id_pedido) -M2/IContratoComercios:: obtenerDatosFiscalesComercio(id_comercio) - M1/IContratoClientes:: obtenerCliente(id_cliente) - Evento publicado: M6\Events\FacturaGenerada(id_factura, monto, xml) (consumido por M7)

Debido a lo grande de el diagrama, se puede encontrar mejor en pacifico-online/docs/actas-subgrupos/SG6_Pagos_Y_Facturacion_Electronica/Diagramas_PC3

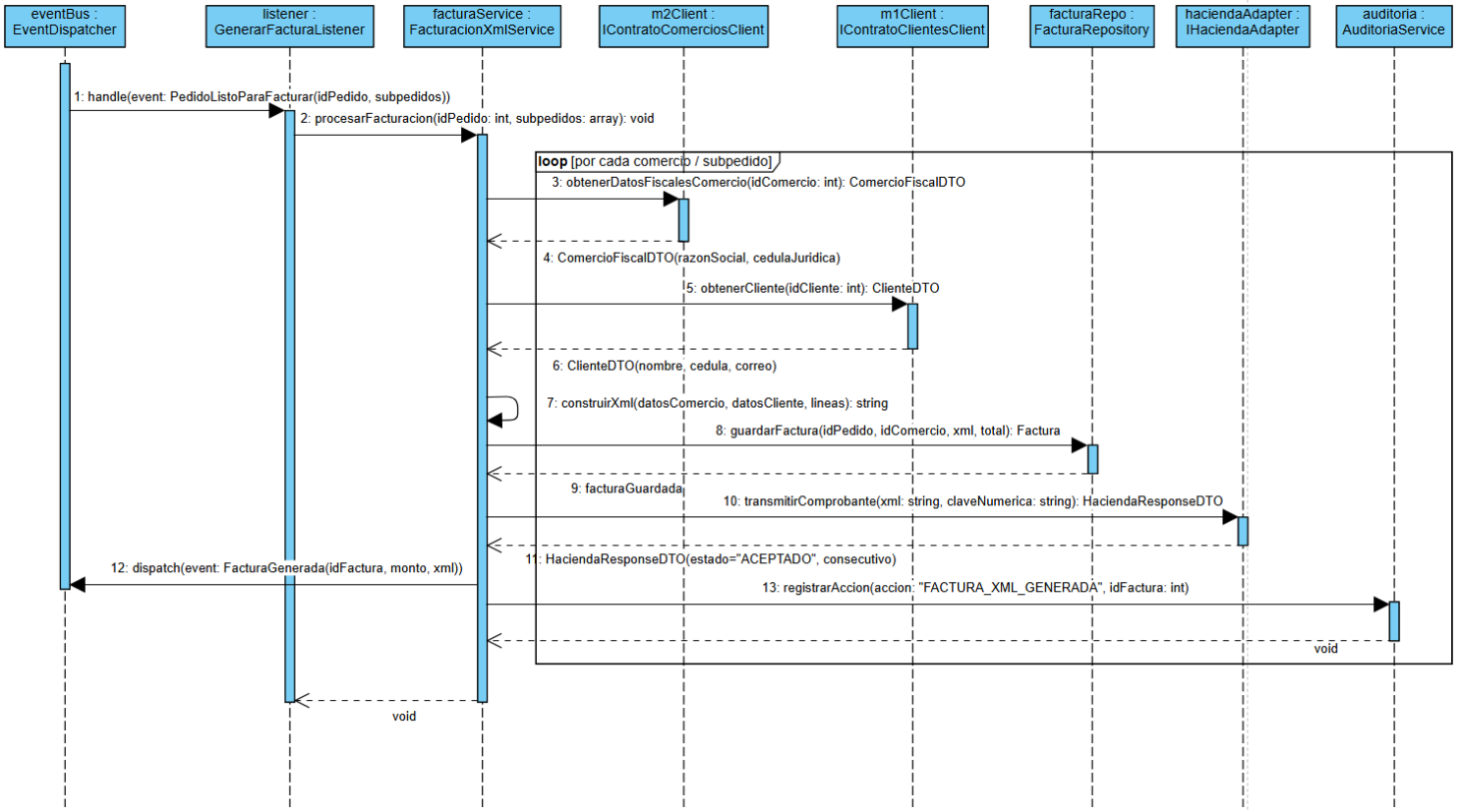
DS-01: CU-001 - Procesar pago del pedido



DS-02: CU-002 - Confirmar pago e Idempotencia



DS-03: CU-004 - Generar factura electrónica XML



6. Modelo de datos del módulo

Modelo en tercera forma normal, implementado como migraciones de Laravel, con restricciones de unicidad y verificación que implementan las reglas de negocio. Se entrega además el SQL resultante.

Cómo hacerlo

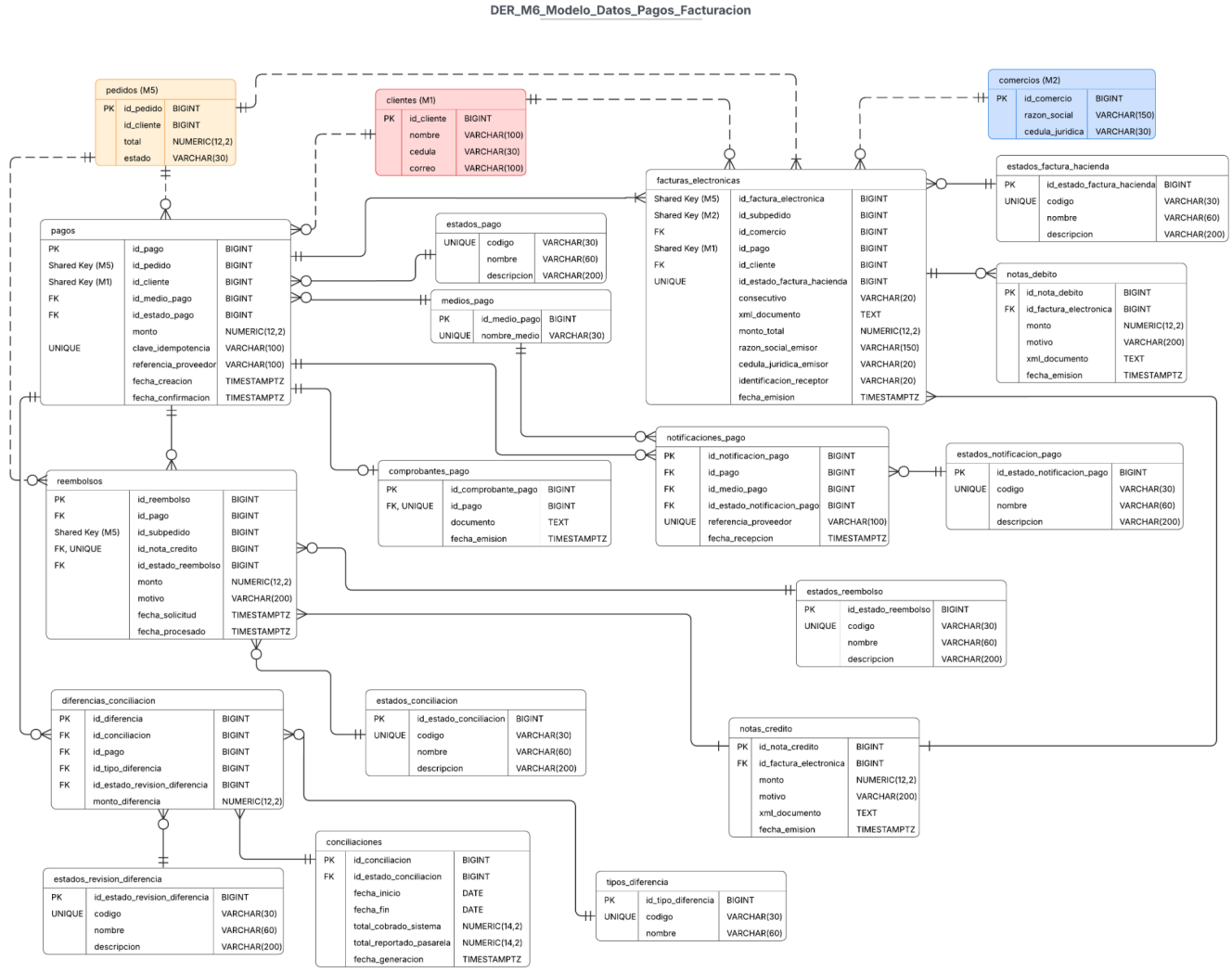
- Usar solo las tablas reservadas por el módulo en el PC2, con los nombres corregidos (minúscula, sin tildes, guión bajo) y las claves con formato id_entidad.
- Las referencias a entidades de otros módulos se guardan solo como clave compartida, según la decisión del Comité (1.9) sobre llaves foráneas entre módulos.
- Revisar cada tabla contra 1FN, 2FN y 3FN y justificar el resultado.
- Cada regla de negocio que se pueda expresar en la base de datos se implementa como restricción (unique, check, not null, foreign key interna) dentro de la migración.
- Para obtener el SQL resultante se puede usar php artisan migrate --pretend, o exportar el esquema de la base de datos una vez aplicadas las migraciones.

6.1 Tablas del módulo

Tabla	Clave primaria	Claves compartidas que guarda	Descripción
estados_pago	id_estado_pago	—	Catálogo de estados válidos para los intentos de pago (pendiente, confirmado, rechazado).
estados_notificacion_pago	id_estado_notificacion_pago	—	Catálogo de estados de notificaciones de pasarelas (recibida, procesada, duplicada, rechazada).
estados_factura_hacienda	*id_estado_factura_hacienda*	—	*Catálogo de estados ante Hacienda (pendiente_envio, enviada, aceptada, rechazada).*
estados_reembolso	*id_estado_reembolso*	—	*Catálogo de estados de reembolsos (solicitado, procesado, rechazado).*
estados_conciliacion	*id_estado_conciliacion*	—	*Catálogo de estados de conciliación periódica (en_proceso, parcial, completada).*
estados_revision_diferencia	*id_estado_revision_diferencia*	—	*Catálogo de estados de revisión de diferencias (pendiente_revision, resuelta).*
tipos_diferencia	*id_tipo_diferencia*	—	*Catálogo de tipos de discrepancias en conciliación (monto_distinto, faltante_en_pasarela, faltante_en_sistema).*
medios_pago	*id_medio_pago*	—	*Catálogo de medios de pago soportados (tarjeta, sinpe, paypal).*
pagos	*id_pago*	*id_pedido (M5), id_cliente (M1)*	*Registra cada intento de cobro sobre un pedido, validando montos e idempotencia inicial.*
notificaciones_pago	*id_notificacion_pago*	—	*Registra los webhooks/avisos de pasarelas externas para asegurar la idempotencia (RN-002).*
comprobantes_pago	*id_comprobante_pago*	—	*Almacena el comprobante o recibo de pago generado para el cliente y comercio.*
facturas_electronicas	*id_factura_electronica*	id_subpedido (M5), id_comercio (M2), id_cliente (M1)	*Comprobante fiscal electrónico (XML) por subpedido/comercio con snapshot fiscal del emisor.*
notas_credito	*id_nota_credito*	—	*Documento que revierte total o parcialmente una factura electrónica previa (RN-004).*
notas_debito	*id_nota_debito*	—	*Documento que ajusta al alza una factura electrónica aceptada por Hacienda (RN-005).*
reembolsos	*id_reembolso*	id_subpedido (M5)	*Gestión de devolución de dinero por cancelación o devolución autorizada (RN-006).*

Tabla	Clave primaria	Claves compartidas que guarda	Descripción
conciliaciones	*id_conciliacion*	—	*Cabecera de la conciliación periódica de pagos frente a reportes de pasarelas y M7.*
diferencias_conciliacion	*id_diferencia*	—	*Detalle de pagos con discrepancias detectadas durante la conciliación (RN-007).*

Debido a lo grande de el diagrama, se puede encontrar mejor en pacifico-online/docs/actas-subgrupos/SG6_Pagos_Y_Facturacion_Electronica/Diagramas_PC3



6.2 Verificación de tercera forma normal

Tabla	1FN (valores atómicos)	2FN (sin dependencias parciales)	3FN (sin dependencias transitivas)	Observaciones
estados_pago	Cumple	Cumple	Cumple	Valores atómicos; clave primaria simple (id_estado_pago) sin dependencias parciales o transitivas.
estados_notificacion_pago	Cumple	Cumple	Cumple	Valores atómicos; clave primaria simple (id_estado_notificacion_pago).
estados_factura_hacienda	Cumple	Cumple	Cumple	Valores atómicos; clave primaria simple (id_estado_factura_hacienda).
estados_reembolso	Cumple	Cumple	Cumple	Valores atómicos; clave primaria simple (id_estado_reembolso).
estados_conciliacion	Cumple	Cumple	Cumple	Valores atómicos; clave primaria simple (id_estado_conciliacion).
estados_revision_diferencia	Cumple	Cumple	Cumple	Valores atómicos; clave primaria simple (id_estado_revision_diferencia).
tipos_diferencia	Cumple	Cumple	Cumple	Valores atómicos; clave primaria simple (id_tipo_diferencia).
medios_pago	Cumple	Cumple	Cumple	Valores atómicos; clave primaria simple (id_medio_pago).
pagos	Cumple	Cumple	Cumple	Atributos atómicos; no existen dependencias parciales al poseer una única PK (id_pago), ni transitivas respecto a las FKs de catálogo.
notificaciones_pago	Cumple	Cumple	Cumple	Llave primaria independiente (id_notificacion_pago); referencia a pagos y catálogos sin duplicar atributos transaccionales.
comprobantes_pago	Cumple	Cumple	Cumple	Relación estricta 1 a 1 con pagos a través de id_pago con restricción UNIQUE, sin dependencias cruzadas.
facturas_electronicas	Cumple	Cumple	Cumple	Atributos atómicos. Incorpora snapshots fiscales del emisor (razon_social_emisor, cedula_juridica_emisor) para evitar dependencias transitivas si el comercio edita sus datos en M2.
notas_credito	Cumple	Cumple	Cumple	Dependencia funcional directa de id_factura_electronica. Montos y motivos totalmente atómicos.
notas_debito	Cumple	Cumple	Cumple	Dependencia funcional directa de id_factura_electronica. Montos y motivos totalmente atómicos.
reembolsos	Cumple	Cumple	Cumple	Depende de id_pago y enlaza el subpedido y una única nota de crédito (UNIQUE en id_nota_credito) sin duplicar datos fiscales.

Tabla	1FN (valores atómicos)	2FN (sin dependencias parciales)	3FN (sin dependencias transitivas)	Observaciones
conciliaciones	Cumple	Cumple	Cumple	Cabecera de periodo con atributos atómicos y PK independiente (id_conciliacion).
diferencias_conciliacion	Cumple	Cumple	Cumple	Detalle relacional que desglosa las discrepancias por pago sin duplicar información de la conciliación o del pago original.

6.3 Restricciones que implementan reglas de negocio

Regla de negocio (PC2)	Tabla	Tipo de restricción	Definición	Nombre en la migración
RN-001: Coincidencia de monto	pagos	CHECK	monto > 0 (La validación estricta de coincidencia con el total de M5 se gestiona en la capa de control y servicio)	pagos_monto_check
RN-002: Idempotencia de cobro	pagos	UNIQUE	((clave_idempotencia))	pagos_clave_idempotencia_unique
RN-002: Idempotencia de notificación	notificaciones_pago	UNIQUE	(referencia_proveedor)	notificaciones_pago_referencia_proveedor_unique
RN-003: Factura solo con pago confirmado	facturas_electronicas	FOREIGN KEY / SERVICE	Validación en servicio del estado del pago + llave foránea interna	facturas_electronicas_id_pago_foreign
RN-004: Nota de crédito menor o igual a factura	notas_credito	CHECK / SERVICE	monto > 0 (La validación del tope máximo frente a la factura original se ejecuta en el método NotaDebito.validarMonto)	notas_credito_monto_check
RN-005: Nota de débito sobre factura aceptada	notas_debito	SERVICE	Validación lógica del estado aceptada de Hacienda en la emisión	N/A (Validación lógica en NotaDebito)
RN-006: Reembolso con pago confirmado y monto válido	reembolsos	CHECK	monto > 0	reembolsos_monto_check
RN-007: Registro obligatorio de diferencias	diferencias_conciliacion	CHECK	monto_diferencia <> 0	diferencias_conciliacion_monto_diferencia_check

6.4 Migraciones

Archivo de migración	Tabla	Descripción
database/migrations/2026_10_01_000001_create_m6_estados_catalogos_table.php	estados_pago, estados_notificacion_pago, estados_factura_hacienda, estados_reembolso, estados_conciliacion, estados_revision_diferencia, tipos_diferencia, medios_pago	Crea las 7 tablas de catálogo y la tabla medios_pago, e incluye las sentencias INSERT (seeder inicial) para poblar los valores base requeridos por las llaves foráneas.
database/migrations/2026_10_01_000002_create_pagos_table.php	pagos	Tabla transaccional para cobros con restricciones de monto positivo, llaves compartidas a M5/M1 e índice de idempotencia clave_idempotencia.

Archivo de migración	Tabla	Descripción
database/migrations/2026_10_01_000003_create_notificaciones_pago_table.php	notificaciones_pago	Almacén de webhooks de pasarelas con índice único en referencia_proveedor para asegurar la idempotencia (RN-002).
database/migrations/2026_10_01_000004_create_comprobantes_pago_table.php	comprobantes_pago	Relación 1 a 1 estricta con pagos para almacenar los recibos emitidos.
database/migrations/2026_10_01_000005_create_facturas_electronicas_table.php	facturas_electronicas	Comprobantes XML fiscales con unificación del campo id_subpedido (M5) y snapshot fiscal del emisor.
database/migrations/2026_10_01_000006_create_notas_credito_debito_table.php	notas_credito, notas_debito	Tablas de ajustes fiscales para reversiones y cargos adicionales (RN-004, RN-005).
database/migrations/2026_10_01_000007_create_reembolsos_table.php	reembolsos	Administración de devoluciones incorporando id_subpedido y la restricción UNIQUE (id_nota_credito).
2026_10_01_000008_create_conciliaciones_tables	conciliaciones, diferencias_conciliacion	Cabeceras de periodos conciliados y detalle de discrepancias (RN-007).

6.5 SQL resultante

```
-- ========================================================================
-- MÓDULO M6: PAGOS Y FACTURACIÓN ELECTRÓNICA - SCRIPT SQL (3FN)
-- ========================================================================

-- 1. TABLAS DE CATÁLOGO Y MEDIOS DE PAGO
CREATE TABLE estados_pago (
   id_estado_pago BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
   codigo VARCHAR(30) NOT NULL UNIQUE,
   nombre VARCHAR(60) NOT NULL,
   descripcion VARCHAR(200)
);

CREATE TABLE estados_notificacion_pago (
   id_estado_notificacion_pago BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
   codigo VARCHAR(30) NOT NULL UNIQUE,
   nombre VARCHAR(60) NOT NULL,
   descripcion VARCHAR(200)
);

CREATE TABLE estados_factura_hacienda (
   id_estado_factura_hacienda BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
   codigo VARCHAR(30) NOT NULL UNIQUE,
   nombre VARCHAR(60) NOT NULL,
   descripcion VARCHAR(200)
);

CREATE TABLE estados_reembolso (
   id_estado_reembolso BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
   codigo VARCHAR(30) NOT NULL UNIQUE,
   nombre VARCHAR(60) NOT NULL,
   descripcion VARCHAR(200)
);

CREATE TABLE estados_conciliacion (
```

```
    id_estado_conciliacion BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    codigo VARCHAR(30) NOT NULL UNIQUE,
    nombre VARCHAR(60) NOT NULL,
    descripcion VARCHAR(200)
);

CREATE TABLE estados_revision_diferencia (
    id_estado_revision_diferencia BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    codigo VARCHAR(30) NOT NULL UNIQUE,
    nombre VARCHAR(60) NOT NULL,
    descripcion VARCHAR(200)
);

CREATE TABLE tipos_diferencia (
    id_tipo_diferencia BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    codigo VARCHAR(30) NOT NULL UNIQUE,
    nombre VARCHAR(60) NOT NULL,
    descripcion VARCHAR(200)
);

CREATE TABLE medios_pago (
    id_medio_pago BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    nombre_medio VARCHAR(30) NOT NULL UNIQUE CHECK (nombre_medio IN ('tarjeta', 'sinpe', 'paypal'))
);

-- SEEDER INICIAL DE CATÁLOGOS
INSERT INTO estados_pago (codigo, nombre, descripcion) VALUES
('pendiente', 'Pendiente', 'Cobro enviado al proveedor, sin confirmación.'),
('confirmado', 'Confirmado', 'El proveedor acreditó el pago exitosamente.'),
('rechazado', 'Rechazado', 'El proveedor rechazó el cobro.');

INSERT INTO estados_notificacion_pago (codigo, nombre, descripcion) VALUES
('recibida', 'Recibida', 'Notificación recibida, pendiente de procesamiento.'),
('procesada', 'Procesada', 'Notificación validada y aplicada al pago.'),
('duplicada', 'Duplicada', 'Referencia ya procesada previamente (idempotencia).'),
('rechazada', 'Rechazada', 'Intento inexistente o firma no válida.');

INSERT INTO estados_factura_hacienda (codigo, nombre, descripcion) VALUES
('pendiente_envio', 'Pendiente de Envío', 'XML generado, pendiente de transmisión.'),
('enviada', 'Enviada', 'Transmitida a Hacienda, a la espera de respuesta.'),
('aceptada', 'Aceptada', 'Aceptada por Hacienda (habilita notas de débito).'),
('rechazada', 'Rechazada', 'Rechazada por las reglas de validación de Hacienda.');

INSERT INTO estados_reembolso (codigo, nombre, descripcion) VALUES
('solicitado', 'Solicitado', 'Reembolso solicitado y en trámite.'),
('procesado', 'Procesado', 'Dinero devuelto y nota de crédito emitida.'),
('rechazado', 'Rechazado', 'Rechazado por incumplimiento de reglas de negocio.');

INSERT INTO estados_conciliacion (codigo, nombre, descripcion) VALUES
('en_proceso', 'En Proceso', 'Comparando transacciones del periodo.'),
('parcial', 'Parcial', 'Falta confirmación de asiento contable de M7.'),
('completada', 'Completada', 'Periodo conciliado por completo.');

INSERT INTO estados_revision_diferencia (codigo, nombre, descripcion) VALUES
('pendiente_revision', 'Pendiente de Revisión', 'Discrepancia a la espera de verificación manual.'),
('resuelta', 'Resuelta', 'Diferencia verificada y solventada por el administrador.');
```

```
INSERT INTO tipos_diferencia (codigo, nombre, descripcion) VALUES
('monto_distinto', 'Monto Distinto', 'Pago registrado en ambos lados pero con importes diferentes.'),
('faltante_en_pasarela', 'Faltante en Pasarela', 'Existe en sistema local pero no en reporte de pasarela.'),
('faltante_en_sistema', 'Faltante en Sistema', 'Reportado por pasarela pero ausente en registros locales.');

INSERT INTO medios_pago (nombre_medio) VALUES ('tarjeta'), ('sinpe'), ('paypal');

-- 2. TABLA TRANSACCIONAL PRINCIPAL: PAGOS
CREATE TABLE pagos (
   id_pago BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
   id_pedido BIGINT NOT NULL,
   id_cliente BIGINT NOT NULL,
   id_medio_pago BIGINT NOT NULL,
   id_estado_pago BIGINT NOT NULL,
   monto NUMERIC(12,2) NOT NULL CHECK (monto > 0),
   clave_idempotencia VARCHAR(100) NOT NULL UNIQUE,
   referencia_proveedor VARCHAR(100),
   fecha_creacion TIMESTAMPTZ NOT NULL DEFAULT NOW(),
   fecha_confirmacion TIMESTAMPTZ,
   CONSTRAINT fk_pagos_medio FOREIGN KEY (id_medio_pago) REFERENCES medios_pago(id_medio_pago),
   CONSTRAINT fk_pagos_estado FOREIGN KEY (id_estado_pago) REFERENCES estados_pago(id_estado_pago)
);

-- 3. TABLA NOTIFICACIONES DE PAGO
CREATE TABLE notificaciones_pago (
   id_notificacion_pago BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
   id_pago BIGINT NOT NULL,
   id_medio_pago BIGINT NOT NULL,
   id_estado_notificacion_pago BIGINT NOT NULL,
   referencia_proveedor VARCHAR(100) NOT NULL UNIQUE,
   fecha_recepcion TIMESTAMPTZ NOT NULL DEFAULT NOW(),
   CONSTRAINT fk_notif_pago FOREIGN KEY (id_pago) REFERENCES pagos(id_pago),
   CONSTRAINT fk_notif_medio FOREIGN KEY (id_medio_pago) REFERENCES medios_pago(id_medio_pago),
   CONSTRAINT fk_notif_estado FOREIGN KEY (id_estado_notificacion_pago) REFERENCES
estados_notificacion_pago(id_estado_notificacion_pago)
);

-- 4. TABLA COMPROBANTES DE PAGO
CREATE TABLE comprobantes_pago (
   id_comprobante_pago BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
   id_pago BIGINT NOT NULL UNIQUE,
   documento TEXT NOT NULL CHECK (length(trim(documento)) > 0),
   fecha_emision TIMESTAMPTZ NOT NULL DEFAULT NOW(),
   CONSTRAINT fk_comprobante_pago FOREIGN KEY (id_pago) REFERENCES pagos(id_pago)
);

-- 5. TABLA FACTURAS ELECTRÓNICAS
CREATE TABLE facturas_electronicas (
   id_factura_electronica BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
   id_subpedido BIGINT NOT NULL,
   id_comercio BIGINT NOT NULL,
   id_pago BIGINT NOT NULL,
   id_cliente BIGINT NOT NULL,
   id_estado_factura_hacienda BIGINT NOT NULL,
   consecutivo VARCHAR(20) NOT NULL UNIQUE,
```

```
    xml_documento TEXT NOT NULL CHECK (length(trim(xml_documento)) > 0),
    monto_total NUMERIC(12,2) NOT NULL CHECK (monto_total > 0),
    razon_social_emisor VARCHAR(150) NOT NULL,
    cedula_juridica_emisor VARCHAR(20) NOT NULL,
    identificacion_receptor VARCHAR(20),
    fecha_emision TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    CONSTRAINT fk_factura_pago FOREIGN KEY (id_pago) REFERENCES pagos(id_pago),
    CONSTRAINT fk_factura_hacienda FOREIGN KEY (id_estado_factura_hacienda) REFERENCES
estados_factura_hacienda(id_estado_factura_hacienda)
);

-- 6. TABLAS NOTAS DE CRÉDITO Y DÉBITO
CREATE TABLE notas_credito (
    id_nota_credito BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    id_factura_electronica BIGINT NOT NULL,
    monto NUMERIC(12,2) NOT NULL CHECK (monto > 0),
    motivo VARCHAR(200) NOT NULL CHECK (length(trim(motivo)) > 0),
    xml_documento TEXT NOT NULL CHECK (length(trim(xml_documento)) > 0),
    fecha_emision TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    CONSTRAINT fk_nc_factura FOREIGN KEY (id_factura_electronica) REFERENCES
facturas_electronicas(id_factura_electronica)
);

CREATE TABLE notas_debito (
    id_nota_debito BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    id_factura_electronica BIGINT NOT NULL,
    monto NUMERIC(12,2) NOT NULL CHECK (monto > 0),
    motivo VARCHAR(200) NOT NULL CHECK (length(trim(motivo)) > 0),
    xml_documento TEXT NOT NULL CHECK (length(trim(xml_documento)) > 0),
    fecha_emision TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    CONSTRAINT fk_nd_factura FOREIGN KEY (id_factura_electronica) REFERENCES
facturas_electronicas(id_factura_electronica)
);

-- 7. TABLA REEMBOLSOS
CREATE TABLE reembolsos (
    id_reembolso BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    id_pago BIGINT NOT NULL,
    id_subpedido BIGINT NULL,
    id_nota_credito BIGINT NULL UNIQUE,
    id_estado_reembolso BIGINT NOT NULL,
    monto NUMERIC(12,2) NOT NULL CHECK (monto > 0),
    motivo VARCHAR(200) NOT NULL CHECK (length(trim(motivo)) > 0),
    fecha_solicitud TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    fecha_procesado TIMESTAMPTZ,
    CONSTRAINT fk_reembolso_pago FOREIGN KEY (id_pago) REFERENCES pagos(id_pago),
    CONSTRAINT fk_reembolso_nc FOREIGN KEY (id_nota_credito) REFERENCES notas_credito(id_nota_credito),
    CONSTRAINT fk_reembolso_estado FOREIGN KEY (id_estado_reembolso) REFERENCES
estados_reembolso(id_estado_reembolso)
);

-- 8. TABLAS CONCILIACIONES Y DIFERENCIAS
CREATE TABLE conciliaciones (
    id_conciliacion BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    id_estado_conciliacion BIGINT NOT NULL,
    fecha_inicio DATE NOT NULL,
```

```
  fecha_fin DATE NOT NULL CHECK (fecha_fin >= fecha_inicio),
  total_cobrado_sistema NUMERIC(14,2) NOT NULL CHECK (total_cobrado_sistema >= 0),
  total_reportado_pasarela NUMERIC(14,2) NOT NULL CHECK (total_reportado_pasarela >= 0),
  fecha_generacion TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  CONSTRAINT fk_conciliacion_estado FOREIGN KEY (id_estado_conciliacion) REFERENCES
estados_conciliacion(id_estado_conciliacion)
);

CREATE TABLE diferencias_conciliacion (
  id_diferencia BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
  id_conciliacion BIGINT NOT NULL,
  id_pago BIGINT NOT NULL,
  id_tipo_diferencia BIGINT NOT NULL,
  id_estado_revision_diferencia BIGINT NOT NULL,
  monto_diferencia NUMERIC(12,2) NOT NULL CHECK (monto_diferencia <> 0),
  CONSTRAINT fk_dif_conciliacion FOREIGN KEY (id_conciliacion) REFERENCES conciliaciones(id_conciliacion),
  CONSTRAINT fk_dif_pago FOREIGN KEY (id_pago) REFERENCES pagos(id_pago),
  CONSTRAINT fk_dif_tipo FOREIGN KEY (id_tipo_diferencia) REFERENCES tipos_diferencia(id_tipo_diferencia),
  CONSTRAINT fk_dif_revision FOREIGN KEY (id_estado_revision_diferencia) REFERENCES
estados_revision_diferencia(id_estado_revision_diferencia)
);
```

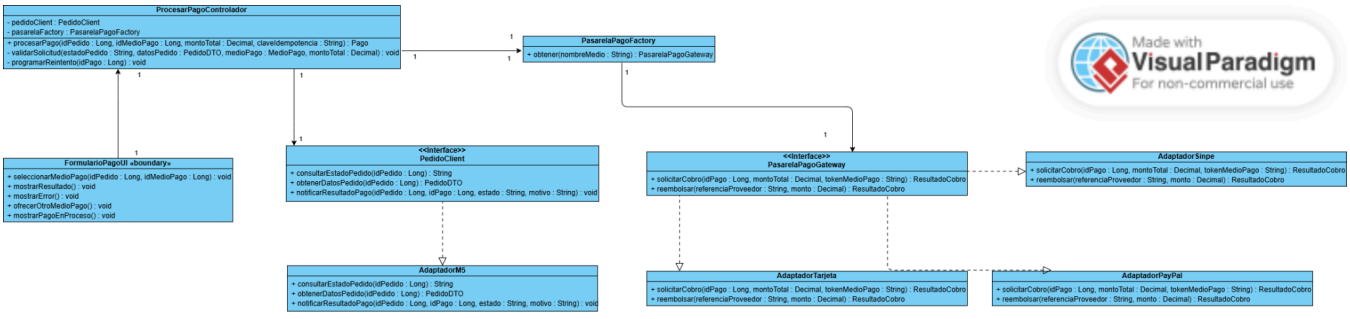
7. Patrones de diseño aplicados (mínimo tres)

Cómo hacerlo

- Aplicar patrones que resuelvan un problema real del módulo, no agregarlos por cumplir: la justificación debe nombrar ese problema.
- Indicar en qué clases del diagrama de diseño (sección 4) se ve cada patrón.
- Verificar que no contradigan los ADR del Comité. Por ejemplo, un patrón para aislar la pasarela de pago o el acceso a otro módulo debe respetar la comunicación por contrato.
- Referencias habituales en Laravel: Repository, Strategy, Adapter, Observer, Factory, Facade, State; se pueden usar otros.

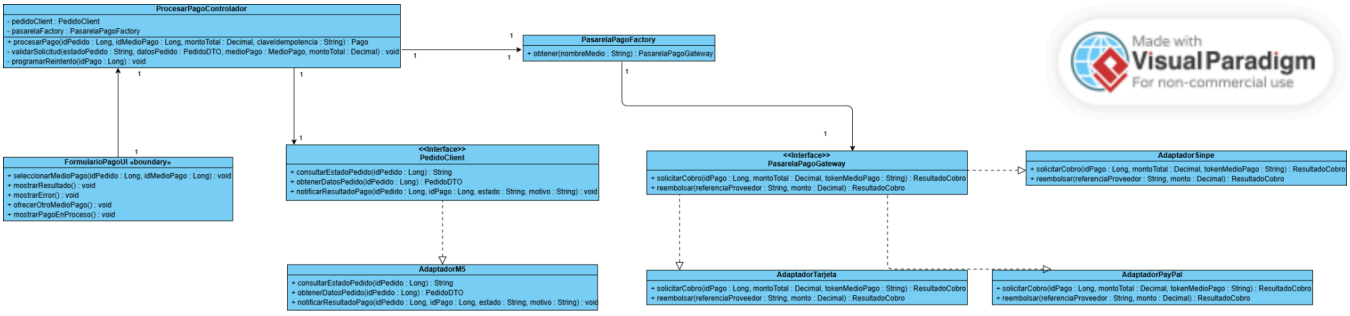
7.1 Strategy

Categoría	*De comportamiento*
Problema del módulo que resuelve	*M6 debe cobrar con tarjeta, SINPE y PayPal. Cada medio se comunica distinto con su proveedor, pero el resto del módulo necesita una sola forma de pedir un cobro y un reembolso. Además, el enunciado deja la pasarela de tarjeta "por determinar" y exige poder sustituirla sin reescribir el módulo*
Dónde se aplica	*Interfaz PasarelaPagoGateway (solicitarCobro y reembolsar), implementada por AdaptadorTarjeta, AdaptadorSinpe y AdaptadorPayPal. La usan ProcesarPagoControlador (cobros) y ReembolsoService (devoluciones), que reciben la estrategia correcta de PasarelaPagoFactory.*
Justificación	*Cambiar o agregar una pasarela es escribir un adaptador nuevo y registrarlo en la fábrica, los controladores no se tocan. Cada proveedor queda aislado tras la interfaz y se trabaja contra simuladores propios .Los tres reciben idPago, que viaja al proveedor como clave de idempotencia. El patrón es interno de M6: no cambia la llamada síncrona de M5 a M6 que define ADR-005.*
Alternativa descartada	*Un if/else por medio de pago dentro del controlador. Se descartó porque cada medio nuevo, o un cambio de pasarela, obligaría a editar el controlador de CU-001 y el de reembolsos, y no se podría probar sin los tres proveedores. Tampoco se crean los adaptadores con new en el controlador, para eso existe la fábrica.*



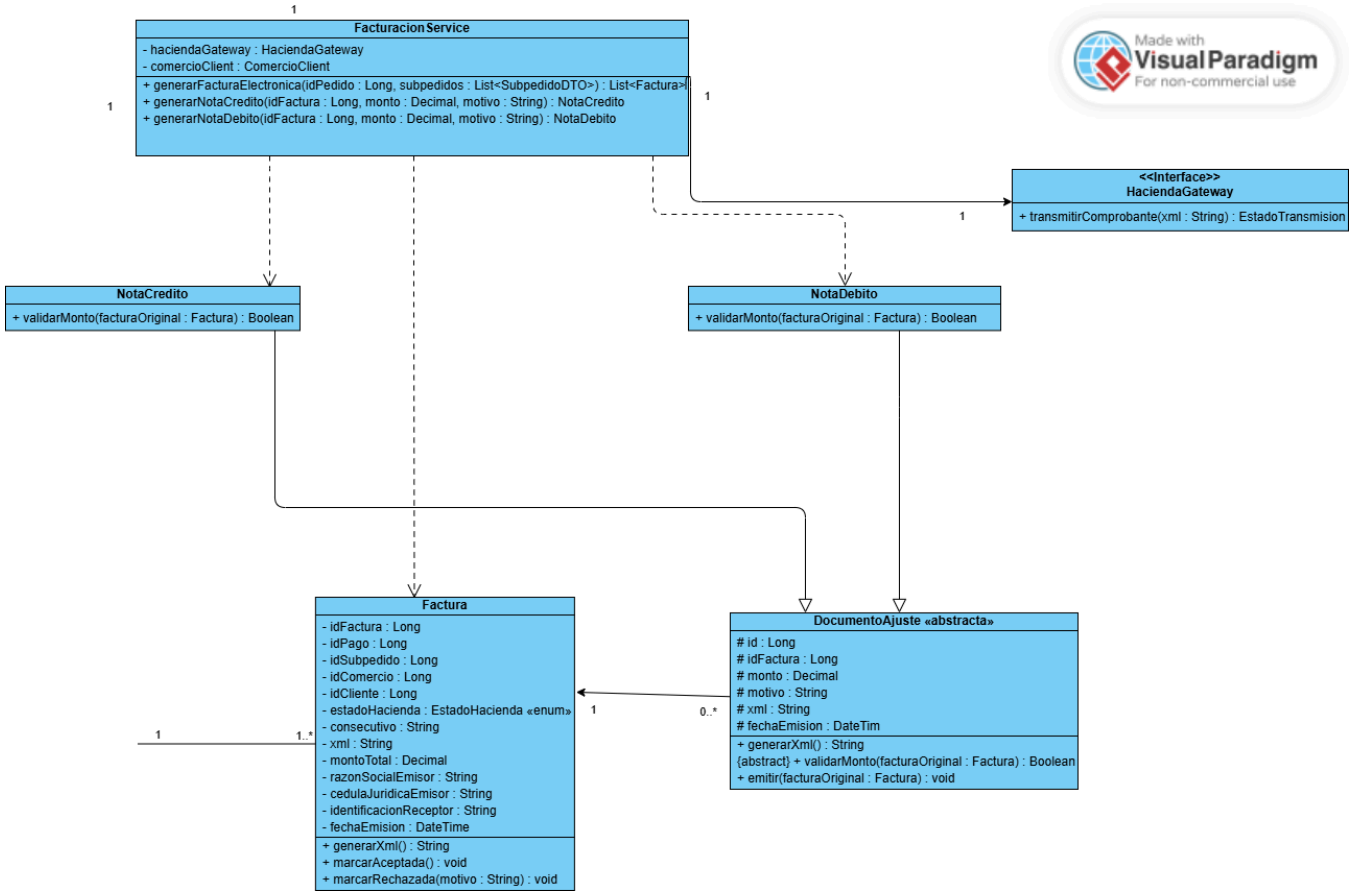
7.2 Adapter

Categoría	*Estructural*
Problema del módulo que resuelve	*M6 necesita consultar el estado y los datos del pedido a M5, consultar el registro contable a M7 y publicar eventos de negocio. La regla de frontera prohíbe leer tablas o usar modelos Eloquent de otro módulo, solo se permite por contrato. Además, los contratos se versionan y pueden cambiar de nombre o de forma.*
Dónde se aplica	*Interfaces PedidoClient, ContabilidadClient y EventPublisher, implementadas por AdaptadorM5, AdaptadorM7 y AdaptadorEventos. ProcesarPagoControlador, ConfirmarPagoControlador, NotificarEventoFinancieroControlador, ReembolsoService y ConciliacionService dependen de las interfaces y nunca de los adaptadores.*
Justificación	*Cada adaptador es el único lugar donde M6 conoce el contrato publicado de otro módulo. Si ese contrato pasa de /v1/ a /v2/ o cambia un nombre de operación , se ajusta un adaptador y los controladores no cambian. Las consultas a M5 ocurren dentro del cobro y los hechos de negocio salen como eventos por AdaptadorEventos, que agrega el id_evento para que M7 descarte duplicados . Como los controladores solo conocen la interfaz, en las pruebas se cambia el adaptador por un doble de prueba y M6 avanza sin esperar a otros subgrupos.*
Alternativa descartada	*Que cada controlador llame directamente al cliente de M5 o M7, o consulte sus tablas. Consultar tablas viola la frontera entre módulos. Llamar directo repite el conocimiento del contrato en cada controlador un cambio de versión obligaría a editarlos todos y no habría dónde colocar un doble de prueba.*



7.3 Template Method

Categoría	*De comportamiento*
Problema del módulo que resuelve	*NotaCredito y NotaDebito se emiten con el mismo proceso validar el monto contra la factura de origen, generar el XML y registrar el documento. Solo cambia la regla de validación, la nota de crédito no puede superar el total de la factura y la de débito exige una factura aceptada por Hacienda.*
Dónde se aplica	*Clase abstracta DocumentoAjuste, con el método plantilla emitir(facturaOriginal), el paso abstracto validarMonto(facturaOriginal) y el paso común generarXml(). Las subclases NotaCredito y NotaDebito implementan validarMonto con su propia regla. FacturacionService solo invoca emitir.*
Justificación	*El orden de los pasos queda fijo en la clase padre, así que ninguna subclase puede emitir sin validar antes. El XML se genera en un solo lugar: si Hacienda cambia el formato, se corrige una vez. Estas dos reglas se comparan contra otra tabla, por lo que no se pueden escribir como restricción CHECK y se validan en el código, cada una dentro de su subclase. Es un patrón interno de M6, no cambia ningún contrato ni la comunicación entre módulos definida en los ADR.*
Alternativa descartada	*Dos clases independientes con el código repetido, o una sola clase con un if según el tipo de nota. Repetir el código obliga a corregir el XML dos veces, y el if mezcla dos reglas en un método que crece con cada tipo nuevo de ajuste.*


FacturacionService
- haciendaGateway : HaciendaGateway
- comercioClient : ComercioClient
+ generarFacturaElectronica(idPedido : Long, subpedidos : List<SubpedidoDTO>) : List<Factura>
+ generarNotaCredito(idFactura : Long, monto : Decimal, motivo : String) : NotaCredito
+ generarNotaDebito(idFactura : Long, monto : Decimal, motivo : String) : NotaDebito
<<Interface>>
HaciendaGateway
+ transmitirComprobante(xml : String) : EstadoTransmision
NotaCredito
+ validarMonto(facturaOriginal : Factura) : Boolean
NotaDebito
+ validarMonto(facturaOriginal : Factura) : Boolean
Factura
- idFactura : Long
- idPago : Long
- idSubpedido : Long
- idComercio : Long
- idCliente : Long
- estadoHacienda : EstadoHacienda «enum»
- consecutivo : String
- xml : String
- montoTotal : Decimal
- razonSocialEmisor : String
- cedulaJuridicaEmisor : String
- identificacionReceptor : String
- fechaEmision : DateTime
+ generarXml() : String
+ marcarAceptada() : void
+ marcarRechazada(motivo : String) : void
DocumentoAjuste «abstracta»
id : Long
idFactura : Long
monto : Decimal
motivo : String
xml : String
fechaEmision : DateTim
+ generarXml() : String
{abstract} + validarMonto(facturaOriginal : Factura) : Boolean
+ emitir(facturaOriginal : Factura) : void
Made with VisualParadigm
For non-commercial use

8. Prototipo de interfaz del módulo, validado

Cómo hacerlo

- Prototipar las pantallas de los casos de uso del módulo, empezando por los realizados en la sección 3.
- Usar React + Tailwind CSS dentro de la carpeta del módulo en el frontend, o una herramienta de prototipado cuyo resultado se pueda llevar a React sin cambios de diseño.
- Cumplir el requerimiento de adaptabilidad: operable desde 360 píxeles de ancho sin desplazamiento horizontal.
- Validar el prototipo con usuarios representativos de los actores del módulo y registrar qué se encontró y qué se cambió.

8.1 Pantallas del prototipo

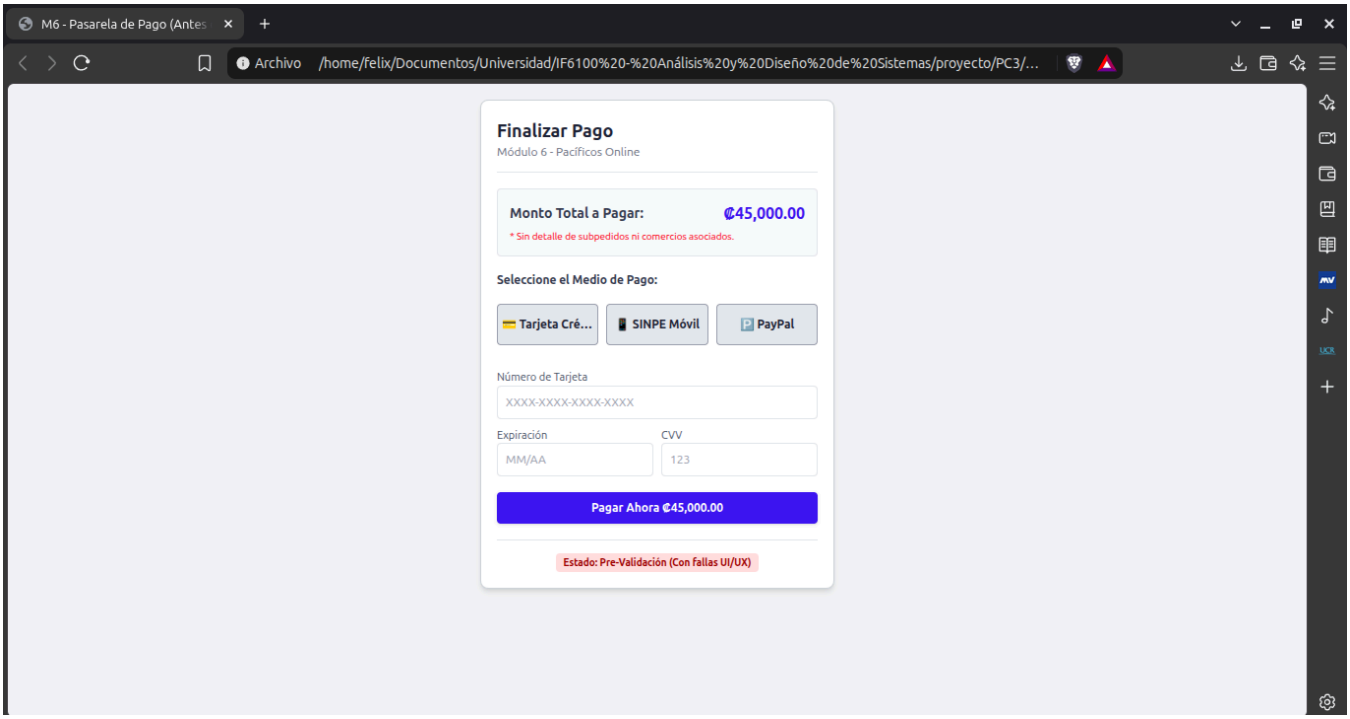
Pantalla	Caso de uso	Actor	Ubicación (repositorio o enlace)
Formulario de Pago y Selección de Medio	CU-001: Procesar pago del pedido	Comprador	frontend/src/modulos/m6-pagos/components/FormularioPago.jsx
Vista de Confirmación de Pago Exitoso/Rechazado	CU-002: Confirmar pago recibido	Comprador / Pasarela	frontend/src/modulos/m6-pagos/components/ResultadoPago.jsx
Detalle y Descarga de Factura Electrónica (XML)	CU-004: Obtener factura electrónica	Comprador / Comercio	frontend/src/modulos/m6-pagos/components/DetalleFactura.jsx
Panel de Historial de Transacciones y Ajustes	CU-008: Historial de pagos y facturas	Administrador de Comercio	frontend/src/modulos/m6-pagos/views/HistorialComercioView.jsx

8.2 Validación

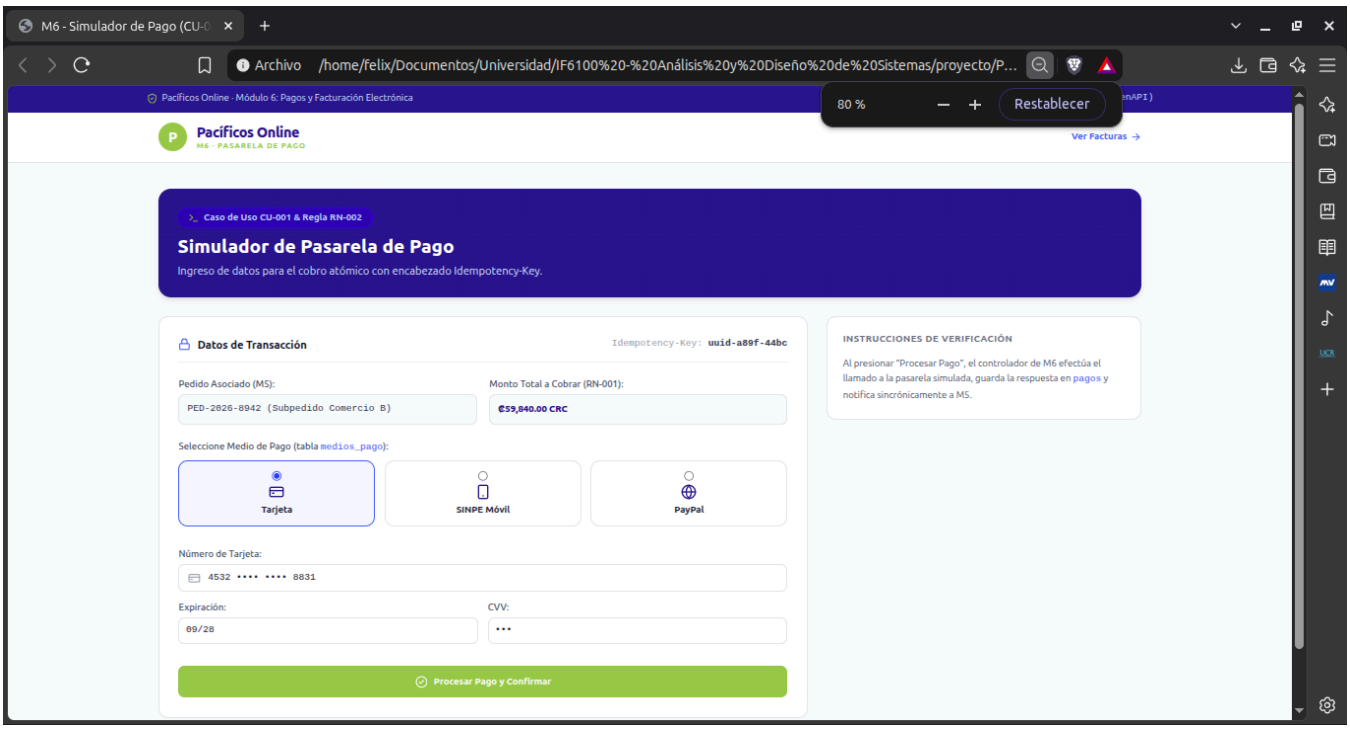
Fecha	21/10/2026
Participantes	2 compradores frecuentes (representantes de M1) y 1 administrador de comercio asociado a M2.
Método	Recorrido guiado de tareas y prueba de adaptabilidad en dispositivos móviles (resolución de 360px de ancho).
Tareas evaluadas	1. Seleccionar un medio de pago (SINPE / Tarjeta) y completar la transacción sin desbordamiento visual. 2. Consultar el estado actual de una factura electrónica emitida.

Hallazgo	Pantalla	Cambio realizado	Estado
En pantallas móviles de 360px, los botones de selección de pasarela se solapaban verticalmente.	Formulario de Pago	Ajuste de clases de Tailwind CSS a grid flexible (grid-cols-1 sm:grid-cols-3) y reducción del padding.	Aplicado
El usuario solicitaba ver de manera más clara el desglose de subpedidos antes de confirmar el cobro total.	Detalle de Factura / Pago	*Incorporación de un acordeón colapsable con los subtotales por subpedido obtenidos de M5.*	*Aplicado*

Captura del antes:



Captura del después:



9. Dobles de prueba de las dependencias externas

Permiten avanzar sin esperar a los otros subgrupos: sustituyen a cada módulo o servicio externo del que depende el módulo.

Cómo hacerlo

- Crear un doble por cada dependencia de la tabla «Lo que el módulo necesita» (1.1), incluidos los servicios externos simulados (pasarela, facturación) si aplican.
- El doble debe respetar la versión del contrato que imita: mismas operaciones, mismos campos y mismos errores.
- Tipos habituales: stub (respuestas fijas), fake (implementación simplificada que funciona), mock (verifica que se hicieron ciertas llamadas).
- Incluir casos de error en los dobles (por ejemplo, existencia insuficiente o pago rechazado), no solo el caso exitoso.
- Definir cómo se cambia del doble a la implementación real sin modificar el código del módulo (por ejemplo, por configuración).

Dependencia	Contrato y versión que imita	Tipo de doble	Escenarios que cubre	Ubicación en el repositorio	Se reemplaza cuando
M1: Clientes (ClientesServiceInterface)	*m1-portal/v1/openapi.yaml*	*fake*	*Éxito: Devuelve perfil de cliente con datos fiscales válidos para facturación (nombre, cédula, correo).* *Error: Simula cliente no encontrado (ERR_CLENTE_NOT_FOUND)*	tests/Doubles/M1/MockClientesService.php	*El módulo M1 publique y despliegue su servicio/API real en el ambiente de integración.*

Dependencia	Contrato y versión que imita	Tipo de doble	Escenarios que cubre	Ubicación en el repositorio	Se reemplaza cuando
M2: Comercios (ComerciosServiceInterface)	*m2-comercios/v1/openapi.yaml*	*Fake*	*Éxito: Retorna razón social, cédula jurídica y datos de facturación electrónica del comercio emisor.* *Error: Simula comercio inactivo o datos fiscales incompletos.*	*[Ruta]tests/Doubles/M2/MockComerciosService.php*	*El módulo M2 despliegue la API de datos fiscales de comercios en el servidor de integración.*
M5: Pedidos (PedidosServiceInterface)	*m5-pedidos/v1/openapi.yaml*	*Fake*	*Éxito: Retorna los detalles de un pedido confirmado con su desglose de sub pedidos y montos.* *Error: Simula pedido no encontrado o con monto inconsistente (PAGO_INVALIDO).*	*tests/Doubles/M5/MockPedidosService.php*	*El módulo 5 tenga disponible la consulta síncrona del contrato de pedidos en el entorno común.*
Pasarela de Pago Externa(PasarelaPagoInterface)	*Adaptador simulador del proveedor (Tarjeta /SINPE/PayPal)*	*Fake/Stub*	*Éxito: Devuelve transacción aprobada con clave de idempotencia (PASARELA-SIM-XXXXX).* *Error: Rechazo de tarjeta, fondos insuficientes o simulación de timeout.*	*app/Services/M6_Pagos/Simuladores/PasarelaPagoSimulador.php*	*Se configure en producción el conector real de la pasarela bancaria/proveedor de pago final.*
Simulador de M6 para terceros (M6_Pagos)	*m6-pagos/v1/openapi.yaml*	*Fake/Mock*	*Éxito: Permite a otros módulos (M1, M5, M7) simular que ejecutan un cobro y reciben respuesta de pago aprobado sin tocar pasarelas reales.*	*tests/Doubles/M6/MockPagosService.php*	*No aplica (se conserva para la suite de pruebas automatizadas de integración de otros subgrupos).*